



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Nulls in Apache Spark – Part 2

In the previous session, we discussed about the following problems in Nulls, and also discussed the first and second problem in detail:

1. What are some common and complex null-related problems?
2. How Spark deals with Nulls?
3. How you can deal with Nulls?

How can we deal with Nulls?

You have three options:

1. Ignore them
2. Remove the null records
3. Fill the null values with some alternative or the best possible value

None of the options are good choices for data analysis.

However, we cannot do anything else, so you must choose one of the three.

The default action is to ignore them and not worry at all until they cause a problem.

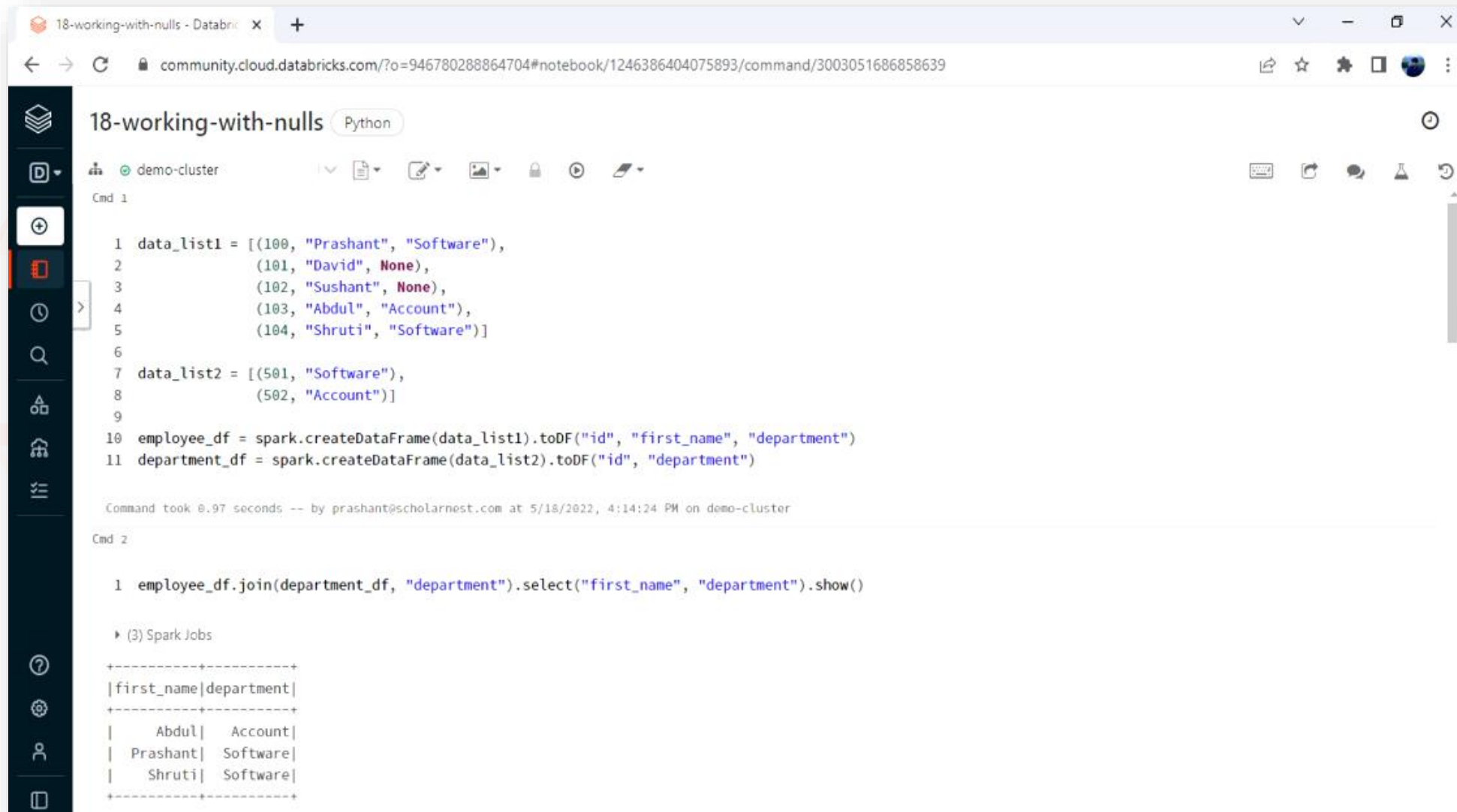
Many people take the ignorance approach, and it works up to some extent.

Why? Why does it work? Because most of the spark transformations and functions are designed to deal with nulls in a particular way.

For example, aggregations such as `sum()`, `avg()`, `min()`, `max()`, etc., leaves the null values aside and do not take them into account.

These functions treat the null records like they do not exist.

Go to your data bricks workspace and open the notebook from the previous lesson. **(Reference : 18-working-with-nulls)**



The screenshot shows a Databricks notebook interface. The browser address bar indicates the URL: `community.cloud.databricks.com/?o=946780288864704#notebook/1246386404075893/command/3003051686858639`. The notebook title is "18-working-with-nulls" and the language is set to "Python". The cluster is "demo-cluster".

Cmd 1

```
1 data_list1 = [(100, "Prashant", "Software"),
2               (101, "David", None),
3               (102, "Sushant", None),
4               (103, "Abdul", "Account"),
5               (104, "Shruti", "Software")]
6
7 data_list2 = [(501, "Software"),
8               (502, "Account")]
9
10 employee_df = spark.createDataFrame(data_list1).toDF("id", "first_name", "department")
11 department_df = spark.createDataFrame(data_list2).toDF("id", "department")
```

Command took 0.97 seconds -- by prashant@scholarnest.com at 5/18/2022, 4:14:24 PM on demo-cluster

Cmd 2

```
1 employee_df.join(department_df, "department").select("first_name", "department").show()
```

▶ (3) Spark Jobs

first_name	department
Abdul	Account
Prashant	Software
Shruti	Software

We have this person's Dataframe.
Now let me calculate the average age from the Dataframe.

18-working-with-nulls Python

demo-cluster

Cmd 3

```
1 person_list = [(100, "Prashant", 30),
2                 (101, "David", None),
3                 (102, "Sushant", None),
4                 (103, "Abdul", 45),
5                 (104, "Shruti", 28)]
6
7 person_df = spark.createDataFrame(person_list).toDF("id", "name", "age")
8 display(person_df)
```

▶ (3) Spark Jobs

	id	name	age
1	100	Prashant	30
2	101	David	null
3	102	Sushant	null
4	103	Abdul	45
5	104	Shruti	28

Showing all 5 rows.

Command took 1.71 seconds -- by prashant@scholarnest.com at 5/18/2022, 4:14:30 PM on demo-cluster

The average age is coming to 34.33.

Cmd 7

```
1 person_df.selectExpr("avg(age)").show()
```

▶ (2) Spark Jobs

avg(age)
34.333333333333336

Command took 1.87 seconds -- by prashant@scholarnest.com at 5/18/2022, 5:50:28 PM on demo-cluster

Now I am removing the null records and then calculating the average age again, and you can see that it is the same as earlier.

So, you can keep the null records in your Dataframe or remove the null records. The `avg()` function returns the same value in both cases. Why? Because the aggregation function in the Dataframe is designed to correct your ignorance and remove the null value records from the computation. And that is why ignoring null works in a lot of cases.

```
Cmd 8

1  person_df.filter("age is not null").selectExpr("avg(age)").show()

▶ (2) Spark Jobs

+-----+
|      avg(age) |
+-----+
|34.33333333333336|
+-----+

Command took 0.69 seconds -- by prashant@scholarnest.com at 5/18/2022, 5:51:31 PM on demo-cluster
```

As discussed earlier, you have three options to work with nulls.

1. Ignore
2. Remove
3. Fill

The ignore will take default implicit action according to the functions and expressions.

But that is not a best practice. You must explicitly remove or fill the null values to avoid unknown problems.

Now let's focus on how to remove or fill the null values. Let's start with the removal.

We have two approaches:

1. `filter()`
2. `na.drop()`

I used a filter condition on the person data frame and took only those records where age is not null. So you can filter out null column records. However, we have more flexible options to drop the null records. The drop options are available via the NA package.

```
Cmd 8

1  person_df.filter("age is not null").selectExpr("avg(age)").show()

▶ (2) Spark Jobs

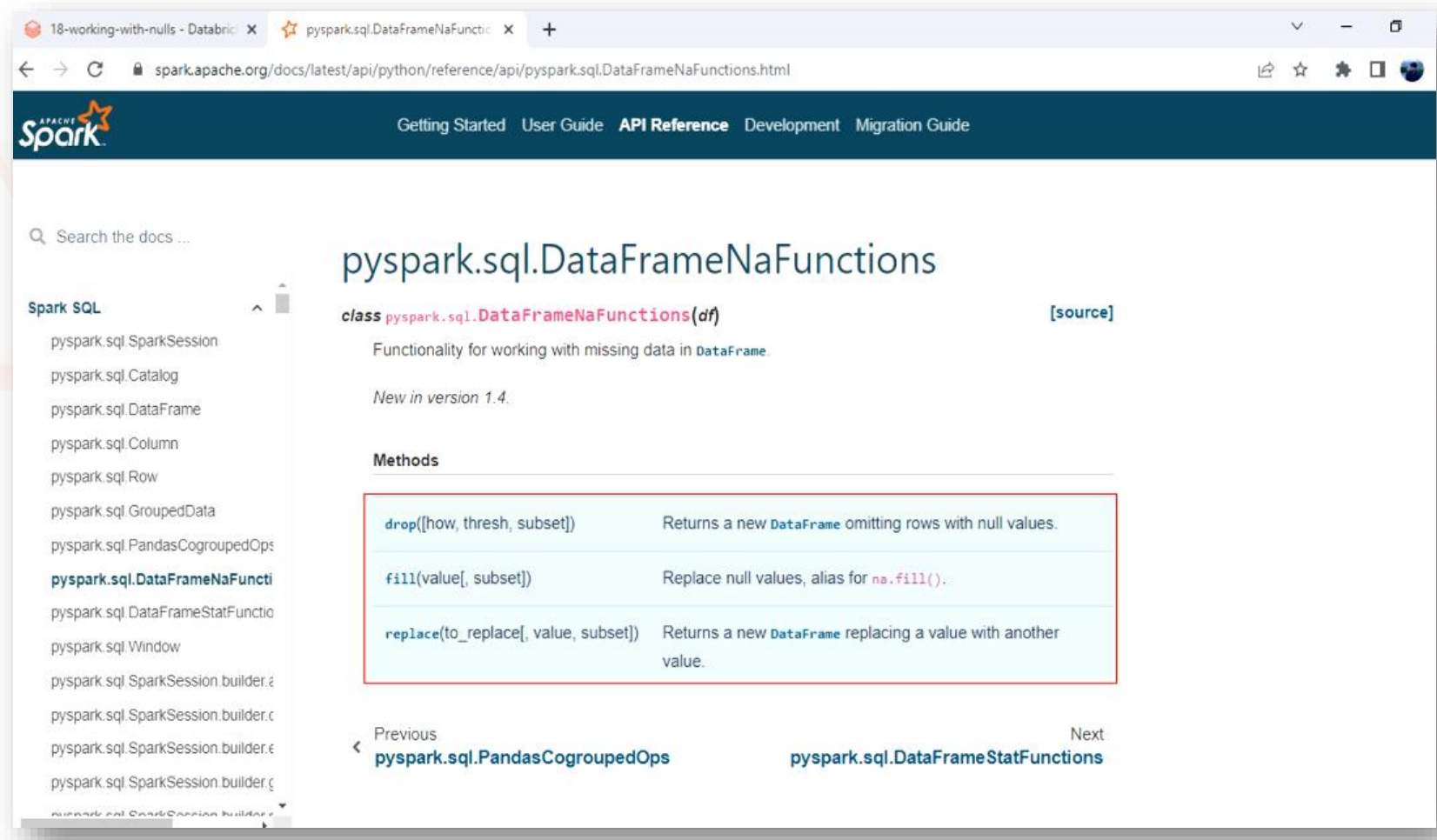
+-----+
|      avg(age) |
+-----+
| 34.3333333333336 |
+-----+

Command took 0.69 seconds -- by prashant@scholarnest.com at 5/18/2022, 5:51:31 PM on demo-cluster

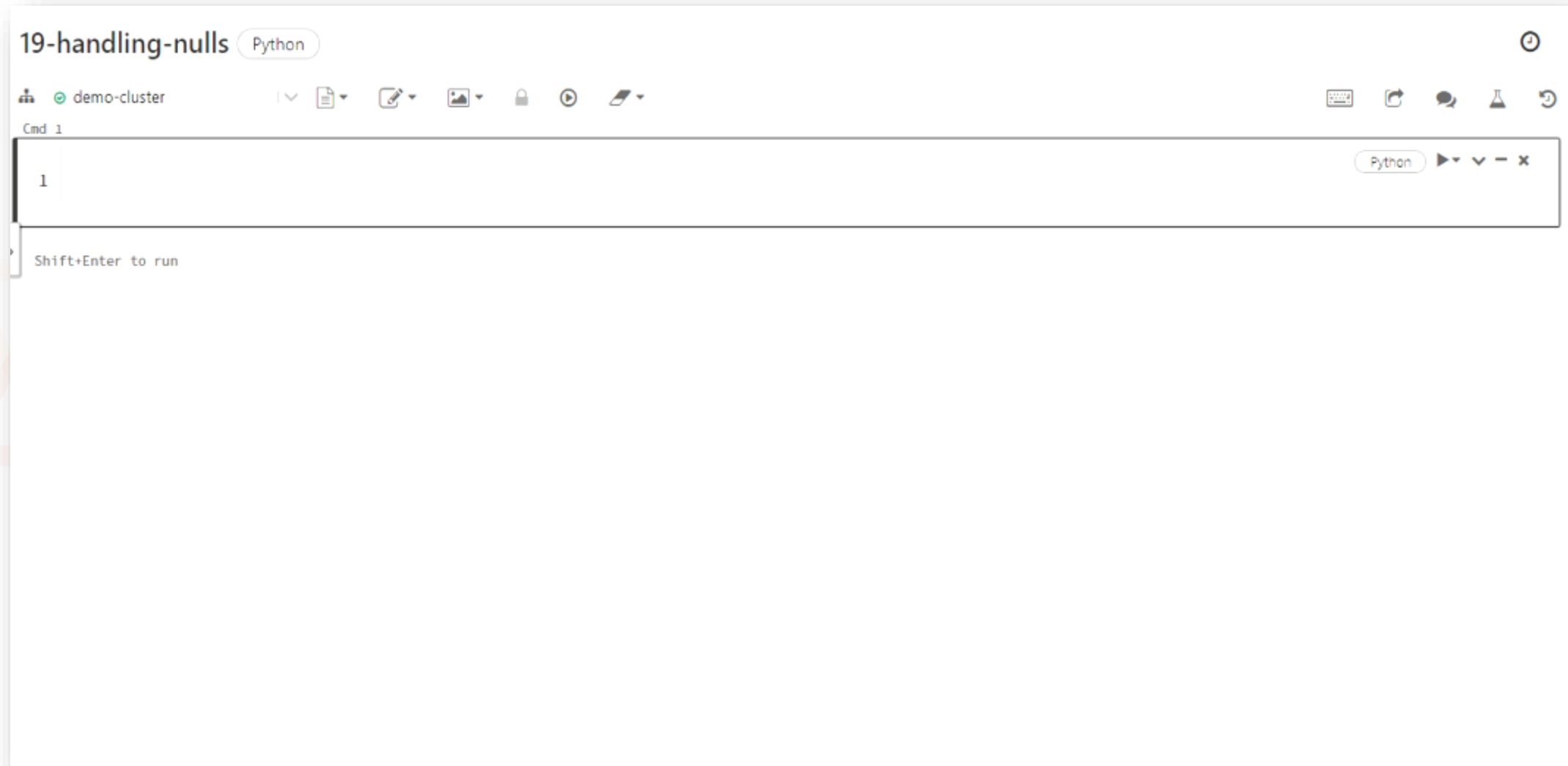
Cmd 9
```

Go to Apache Spark API reference page and search for `pyspark.sql.DataFrameNaFunctions`, and you will see the below page. (Reference : <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.DataFrameNaFunctions.html>)

You will see three functions highlighted below. Now let us use these functions.



I will create a new notebook to explore these functions (**Reference : 19.handling-nulls**)



I have loaded a sample dataset and created a Dataframe. I am loading a CSV data file. In fact, the data file is tab-separated. So it is not a CSV file but a TSV file. However, Spark supports CSV, TSV, and similar files under the CSV format. So you use the CSV format and set an option for the separator. I am loading a tab-separated file. The format is CSV, and the separator is a tab character.

19-handling-nulls Python

demo-cluster

Cmd 1

```
1 song_schema = """artist_id string, artist_latitude double, artist_longitude double,
2     artist_location string, artist_name string, duration double, end_of_fade_in double,
3     key int, key_confidence double, loudness double, release string, song_hotnes double,
4     song_id string, start_of_fade_out double, tempo double, time_signature double,
5     time_signature_confidence double, title string, year double, partial_sequence int"""
6
7 songs_df = spark.read \
8     .format("csv") \
9     .option("sep", "\t") \
10    .schema(song_schema) \
11    .load("/databricks-datasets/songs/data-001/part-00000")
12
13 print(songs_df.count())
14 display(songs_df)
```

Shift+Enter to run

Here is the output Dataframe.

We have 285 records. If you look at the data you can see we have lots of nulls. The artist_latitude and longitude values are null for many records. We want to remove these records from the Dataframe. If you try using the filter method, it would be a difficult task as it would require a long filter condition. I mean, filter where artist_latitude is not null, then filter where artist_longitude is not null. Similarly, you may have to add multiple filter conditions.

19-handling-nulls

Python

demo-cluster

7

songs_df = spark.read \

8

.format("csv") \

9

.option("sep", "\t") \

10

.schema(song_schema) \

11

.load("/databricks-datasets/songs/data-001/part-00000")

12

13

print(songs_df.count())

14

display(songs_df)

(3) Spark Jobs

285

	artist_id	artist_latitude	artist_longitude	artist_location	artist_name
1	AR81V6H1187FB48872	null	null	null	Earl Sixteen
2	ARVVZQP11E2835DBC8	null	null	null	Wavves
3	ARFG9M11187FB3BBCB	null	null	Nashua USA	C-Side
4	ARK4Z2O1187FB45FF0	null	null	null	Harvest
5	AR4VQSG1187FB57E18	35.25082	-91.74015	Searcy, AR	Gossip
6	ARNBV1X1187B996249	null	null	null	Alex
7	ARXOEZX1187B9882A1	null	null	null	Elie Attieh

Showing all 285 rows.

Command took 5.57 seconds -- by prashant@scholarnest.com at 5/18/2022, 6:02:24 PM on demo-cluster

The NA Functions are available with the dot NA notation. The drop() function removes all the rows where any column has a null value. So, the new Dataframe has got only 62 rows. We removed all the rows where any of the columns are null.

19-handling-nulls

Python

demo-cluster

Command took 5.57 seconds -- by prashant@scholarnest.com at 5/18/2022, 6:02:24 PM on demo-cluster

Cmd 2

```
1 clean_df = songs_df.na.drop()
2 print(clean_df.count())
3 display(clean_df)
```

(3) Spark Jobs

62

	artist_id	artist_latitude	artist_longitude	artist_location	artist_name	duration	end_of_fade_in	key
1	ARMLOSJ1187B98B08C	52.08399	4.31741	The Hague, Netherlands	Anouk	266.60526	0.27	4
2	ARLGLB31187B9AE0C2	38.2589	-92.43659	Missouri	Deep Thinkers	237.76608	0	4
3	ARRUHHG11F50C4F353	42.31256	-71.08868	Mass. - Boston	Cheryl Melody	132.96281	0	6
4	ARTC1LV1187B9A4858	51.4536	-0.01802	Goldsmith's College, Lewisham, Lo	The Bonzo Dog Doo Dah Band	189.93587	0.345	10
5	AR0J0JF1187B991270	33.61655	-117.93037	Newport Beach, CA	Chris Wall	241.52771	0.235	7
6	ARSI3101187FB5BBC3	45.51228	-73.55439	Montreal QC	The Besnard Lakes	350.69342	6.629	2
7	ARB2FZP1187B99C739	55.95415	-3.20277	Edinburgh, Scotland	Fire Engines	53.86404	0.235	6

Showing all 62 rows.

Command took 1.59 seconds -- by prashant@scholarnest.com at 5/18/2022, 6:04:15 PM on demo-cluster

The drop() takes three arguments.

1. how -> any or all
2. thresh -> integer
3. subset -> list of column names to consider

The "how" can take any or all. The default value is any.

So the drop() function will remove a record if any of the column values are null. You can try to set it to all, and the drop() function will remove a record only if all the records are null.

I have changed the code and tried it with all values. And you can see that we got 285 records. So nothing is removed because we do not have any record where all the columns are null.

Cmd 2

```
1 clean_df = songs_df.na.drop(how="all")
2 print(clean_df.count())
3 display(clean_df)
```

(3) Spark Jobs

285

	artist_id	artist_latitude	artist_longitude	artist_location	artist_name
1	AR81V6H1187FB48872	null	null	null	Earl Sixteen
2	ARVVZQP11E2835D8CB	null	null	null	Wavves
3	ARFG9M11187FB38BCB	null	null	Nashua USA	C-Side
4	ARK4Z2O1187FB45FF0	null	null	null	Harvest
5	AR4VQSG1187FB57E18	35.25082	-91.74015	Searcy, AR	Gossip
6	ARNBV1X1187B996249	null	null	null	Alex
7	ARXOEZX1187B9882A1	null	null	null	Elie Attieh

Showing all 285 rows.

Command took 1.29 seconds -- by prashant@scholarnest.com at 5/18/2022, 6:06:40 PM on demo-cluster

Now in this code we are using subset of columns. This example is to remove a record if all of these given columns are null. You can also change the how parameter to any, and it will remove a record if any of the given three columns are null. So you can combine the arguments as per your requirement. You can see the output that we are left with the 143 records. So you have all and any options. And you can combine it with the list of columns.

Cmd 2

```
1 clean_df = songs_df.na.drop(how="all", subset=['artist_latitude', 'artist_longitude', 'artist_location'])
2 print(clean_df.count())
3 display(clean_df)
```

▶ (3) Spark Jobs

143

	artist_id	artist_latitude	artist_longitude	artist_location	artist_name	duration	end_of_fac
1	ARFG9M11187FB38BCB	null	null	Nashua USA	C-Side	247.32689	0
2	AR4VQSG1187FB57E18	35.25082	-91.74015	Searcy, AR	Gossip	430.23628	0
3	ARXPUIA1187B9A32F1	null	null	Rome, Italy	Simone Cristicchi	220.00281	2.119
4	ARNPPTH1187B9AD429	51.4855	+0.37196	Heston, Middlesex, England	Jimmy Page	156.86485	0.334
5	ARHAMGQ1187B9B6491	30.66121	+93.89386	Kirbyville, TX	Ivory Joe Hunter	170.39628	0.166
6	ARORFTD1187B99F7D4	null	null	台北, Taiwan	Epitaph	215.95383	5.045
7	AR8UA7Z1187B9AD9C9	48.10751	-1.68447	Rennes	Étienne Daho	303.62077	0

Showing all 143 rows.



Command took 1.33 seconds -- by prashant@scholarnest.com at 5/18/2022, 6:08:16 PM on demo-cluster

We also have a thresh hold option. You can remove the how and use the thresh hold option, and that is what is shown in the screenshot below. So this one says. Look for the given list of columns and remove a record if we have less than two non-null values. The condition is confusing. But it simply means keep the record if we have at least two non-null values amongst these given columns.

Cmd 2

```
1 clean_df = songs_df.na.drop(thresh=2, subset=['artist_latitude', 'artist_longitude', 'artist_location'])
2 print(clean_df.count())
3 display(clean_df)
```

(3) Spark Jobs

103

	artist_id	artist_latitude	artist_longitude	artist_location	artist_name	duration	end_of_fad
1	AR4VQSG1187F857E18	35.25082	-91.74015	Searcy, AR	Gossip	430.23628	0
2	ARNPPTH1187B9AD429	51.4855	-0.37196	Heston, Middlesex, England	Jimmy Page	156.86485	0.334
3	ARHAMGQ1187B9B6491	30.66121	-93.89386	Kirbyville, TX	Ivory Joe Hunter	170.39628	0.166
4	AR8UA7Z1187B9AD9C9	48.10751	-1.68447	Rennes	Étienne Daho	303.62077	0
5	ARMLOSJ1187B98B08C	52.08399	4.31741	The Hague, Netherlands	Anouk	266.60526	0.27
6	ARLGLB31187B9AE0C2	38.2589	-92.43659	Missouri	Deep Thinkers	237.76608	0
7	ARRUHHG11F50C4F353	42.31256	-71.08868	Mass. - Boston	Cherl Melodv	132.96281	0

Showing all 103 rows.

Command took 0.97 seconds -- by prashant@scholarnest.com at 5/18/2022, 6:09:43 PM on demo-cluster

www.scholarnest.com

ScholarNest

© ScholarNest Technologies Pvt Ltd.

So we have a super flexible `na.drop()` function. You can use this function to drop null records explicitly.

However, you do not want to drop your records for having nulls in some scenarios.

Instead, you want to fill those null values and try to repair them.

We have two NA functions for doing that.

1. `fill()`
2. `replace()`

Here is an example that fills the artist_latitude and longitude with zeros. It doesn't make much sense to fill the lat/long values with zeros, but you have an option to fill the nulls with whatever you want.

Cmd 3

```
1 filled_df = songs_df.na.fill({'artist_latitude':0, 'artist_longitude':0})
2 print(filled_df.count())
3 display(filled_df)
```

(3) Spark Jobs

285

	artist_id	artist_latitude	artist_longitude	artist_location	artist_name
1	AR81V6H1187FB48872	0	0	null	Earl Sixteen
2	ARVVZQP11E2835D8CB	0	0	null	Wavves
3	ARFG9M11187FB38BCB	0	0	Nashua USA	C-Side
4	ARK4Z2O1187FB45FF0	0	0	null	Harvest
5	AR4VQSG1187FB57E18	35.25082	-91.74015	Searcy, AR	Gossip
6	ARNBV1X1187B996249	0	0	null	Alex
7	ARXOEZX1187B982A1	0	0	null	Elie Attieh

Showing all 285 rows.

Command took 1.14 seconds -- by prashant@scholarnest.com at 5/18/2022, 6:11:47 PM on demo-cluster

We also have a `replace()` method. But the `replace()` method is for a different purpose. Sometimes, you get data that is already filled with some values that are not meaningful.

Here is a `base_df` shown below. Let's assume you loaded a Dataframe from a CSV file, and the `artist_location` has got UnKnown values. These are not null values. But it is not meaningful, and they represent a lack of know value. You want to change these values with something else. You cannot use the `fill()` method because the `fill()` method fills a null column.

Cmd 4

```
1 base_df = songs_df.na.fill({"artist_location": "UnKnown"})
2 display(base_df)
```

(1) Spark Jobs

	artist_id	artist_latitude	artist_longitude	artist_location	artist_name
1	AR81V6H1187FB48872	null	null	UnKnown	Earl Sixteen
2	ARVVZQP11E2835D8CB	null	null	UnKnown	Wavves
3	ARFG9M11187FB38BCB	null	null	Nashua USA	C-Side
4	ARK4Z2O1187FB45FF0	null	null	UnKnown	Harvest
5	AR4VQSG1187FB57E18	35.25082	-91.74015	Searcy, AR	Gossip
6	ARNBV1X1187B996249	null	null	UnKnown	Alex
7	ARXOEZX1187B9882A1	null	null	UnKnown	Elie Attieh

Showing all 285 rows.



Command took 0.52 seconds -- by prashant@scholarnest.com at 5/18/2022, 6:14:17 PM on demo-cluster

We can use the replace() to change the values as shown below, we look for the UnKnown in the artist location and replace it with the USA.

Cmd 5

```
1 replaced_df = base_df.na.replace(["UnKnown":"USA"], ["artist_location"])
2 display(replaced_df)
```

(1) Spark Jobs

	artist_id	artist_latitude	artist_longitude	artist_location	artist_name
1	AR81V6H1187FB48872	null	null	USA	Earl Sixteen
2	ARVVZQP11E2835DBC8	null	null	USA	Wavves
3	ARFG9M11187FB3BBCB	null	null	Nashua USA	C-Side
4	ARK4Z2O1187FB45FF0	null	null	USA	Harvest
5	AR4VQSG1187FB57E18	35.25082	-91.74015	Searcy, AR	Gossip
6	ARNBV1X1187B996249	null	null	USA	Alex
7	ARXOEZX1187B9882A1	null	null	USA	Elie Attieh

Showing all 285 rows.

Command took 0.49 seconds -- by prashant@scholarnest.com at 5/18/2022, 6:16:46 PM on demo-cluster

You have three NA functions:

1. `drop()`
2. `fill()`
3. `replace()`

These functions can help you to deal with the null columns. The `drop()` allows you to drop the null column records. The `fill()` is to change the null values with some other values. The `replace()` is to change null or other similar values. These three functions are super flexible and take 2-3 parameters. You can customize the use of these functions using those parameters and meet most of your requirements. I cannot create and explain all possible combinations and scenarios. However, you should understand the purpose of these functions and try using them when you have appropriate requirements.

Besides these NA functions, you also have some Spark SQL built-in functions that allow you to change the null values.

Here are some examples:

COALESCE(), NULLIF(), IFNULL(), NVL(), NVL2(), ISNULL(), ISNOTNULL()

These are SQL built-in functions.

You can refer to the documentation and learn some details about these functions.

(Reference : <https://spark.apache.org/docs/latest/api/sql/index.html>)



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com